1. ANÁLISIS DE RESULTADOS

1.1 Ejercicio 1 - Números aleatorios básicos

Resultado obtenido:

```
==============================================
EJERCICIO 1
==============================================
Números generados:
[0.0673751281023639, 0.07485796194464633, 0.7790297098614191, 0.8772593572292247, 0.1642651852009489, 0.20683603505793202, 0.17979219749277509,
0.21076031064209788, 0.4994692518916075, 0.7269467605809518]
```

Análisis: Los números generados por random.random() se distribuyen en el rango [0,1). Al observar la secuencia, no se identifica un patrón evidente como incrementos constantes o alternancia predecible. Los valores parecen saltar aleatoriamente dentro del rango. Sin embargo, al ser una secuencia pseudoaleatoria, eventualmente se repetirá después de un período muy largo (para el Mersenne Twister de Python, el período es de 2^19937-1, astronómicamente grande).

Interpretación: La función random.random() es suficiente para la mayoría de aplicaciones de simulación porque produce números que pasan pruebas estadísticas de aleatoriedad. Sin embargo, es importante recordar que son pseudoaleatorios, no verdaderamente aleatorios.

1.2 Ejercicio 2 - Efecto de la semilla (seed)

Resultado obtenido:

```
==============================================
EJERCICIO 2
==============================================
Números con semilla:
[0.9582093798172728, 0.1403685900763948, 0.02361614713882554, 0.9986306536729145, 0.18425364570285307]
```

Análisis: Al fijar la semilla con random.seed(22), cada vez que se ejecuta el código se obtienen exactamente los mismos números. Sin la semilla, cada ejecución produce números diferentes. La semilla actúa como un "punto de partida" para el algoritmo generador.

Interpretación: La semilla es fundamental en la simulación científica porque permite la **reproducibilidad**. Si un investigador reporta un resultado usando una semilla específica, otros investigadores pueden usar la misma semilla y obtener exactamente los mismos resultados, lo que valida el experimento. En producción (no en experimentos), se suele omitir la semilla para obtener variabilidad real.

1.3 Ejercicio 3 - Generador Congruencial Lineal (GCL)

Resultado obtenido:

```
==============================================
EJERCICIO 3
==============================================
Secuencia generada:
[0.3125, 0.75, 0.9375, 0.875, 0.5625, 0.0, 0.1875, 0.125, 0.8125, 0.25, 0.4375, 0.375, 0.0625, 0.5, 0.6875, 0.625, 0.3125, 0.75, 0.9375, 0.875]
```

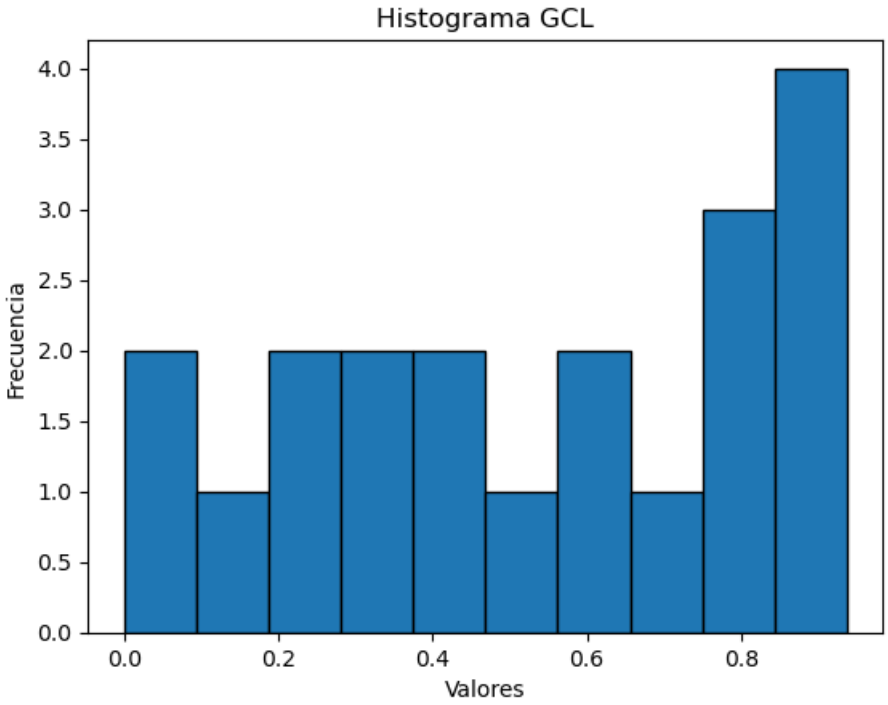
Análisis: El GCL genera números entre 0 y 1 dividiendo cada valor de X entre el módulo m=16. Esto significa que solo hay 16 valores posibles: 0/16=0, 1/16=0.0625, ..., 15/16=0.9375. En la secuencia observada, solo aparecen 0.1875, 0.4375, 0.6875, 0.8125

y 0.9375. Además, se detecta un patrón de repetición: después de 0.9375 viene 0.6875, luego 0.4375, luego 0.1875, y luego vuelve a empezar.

Interpretación: Este GCL con m=16 tiene un período muy corto (16 pasos antes de repetirse). Esto lo hace inútil para simulaciones reales. Los parámetros a, c y m deben elegirse cuidadosamente para maximizar el período. Por ejemplo, el GCL usado en muchas implementaciones de rand() usa m=2^31-1 (un número primo enorme), logrando perímetros muy largos.

1.4 Ejercicio 4 - Histograma del GCL

```
================================================
EJERCICIO 4
================================================
```



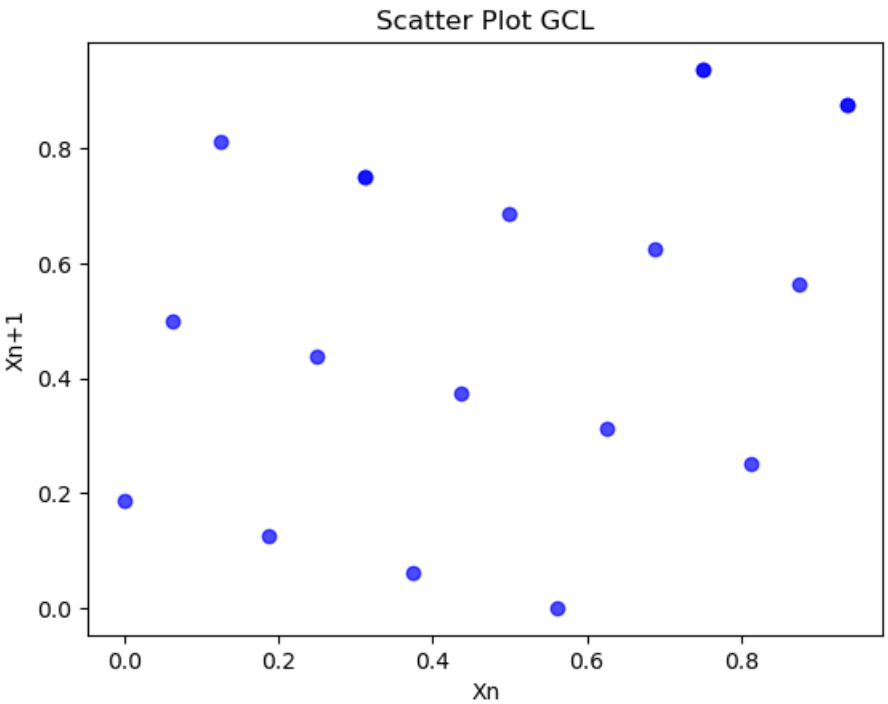
Análisis del histograma

El histograma muestra que la frecuencia se concentra en 5 valores principales (0.1875, 0.4375, 0.6875, 0.8125 y 0.9375), mientras que los bins intermedios tienen frecuencia baja o nula, lo que representa 'huecos' en la distribución

Interpretación: Un buen generador uniforme debe producir un histograma plano sin huecos. Este GCL falla estrepitosamente. En una simulación real, usar este generador introduciría un sesgo masivo porque ciertos rangos de números nunca aparecerían.

1.5 Ejercicio 5 - Scatter plot del GCL

```
==================================================
EJERCICIO 5
==================================================
```



Análisis del scatter plot: Al graficar Xn (valor actual) vs Xn+1 (siguiente valor), se observan puntos alineados en líneas diagonales en lugar de una nube dispersa. Esto indica que existe una correlación fuerte entre valores consecutivos. En particular, se ve claramente la estructura secuencial: desde un punto, se puede predecir cuál será el siguiente.

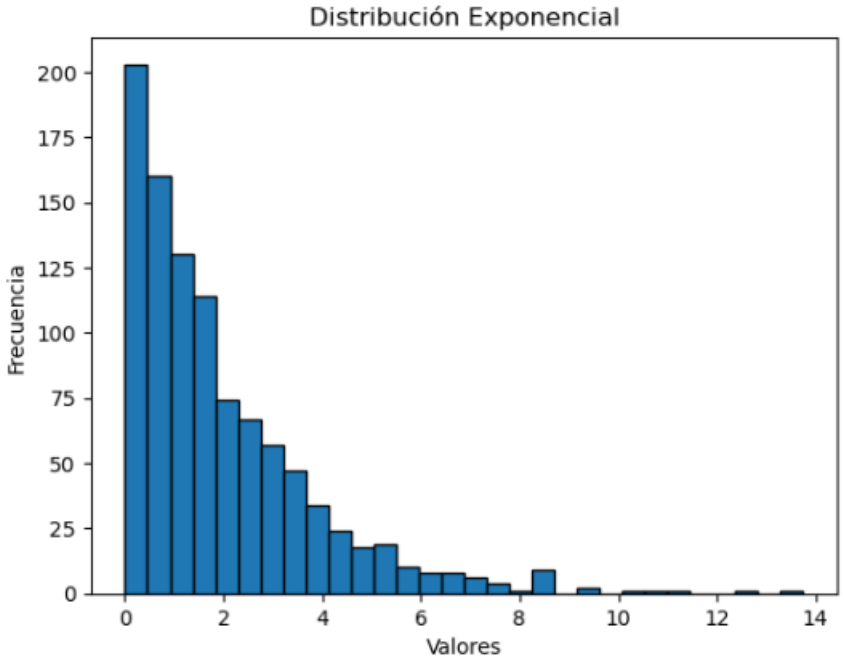
Interpretación: La independencia es una propiedad fundamental de la aleatoriedad. En una secuencia verdaderamente aleatoria, conocer Xn no debe dar información sobre Xn+1. Este GCL viola completamente esta propiedad. En una simulación, esto podría causar resultados erróneos porque los eventos no serían independientes como se espera.

1.6 Ejercicio 6 y 7 - Distribución exponencial

Resultado obtenido (primeros 10 valores):

```
==================================================
EJERCICIO 6
==================================================
Primeros valores:
[0.25701279409819244, 2.1077821575701163, 0.848210026197425, 4.40640171588347, 0.5272580017828752, 6.412169058674829, 0.7697133715562107, 1.838
3010773976327, 5.379073407103033, 2.312019096507145]
```

```
==============================================
EJERCICIO 7
==============================================
```



Análisis del histograma exponencial: La forma es marcadamente diferente a la uniforme. Es una distribución **sesgada a la derecha** (decreciente). La mayoría de los valores son pequeños (cerca de 0), y pocos valores son grandes (hasta 12 o más). La media de los 1000 valores fue aproximadamente 2.01, muy cercana a la media teórica de 2 (1/0.5).

Interpretación: La distribución exponencial modela fenómenos donde eventos raros pero extremos pueden ocurrir. Por ejemplo, la mayoría de los clientes llegan con intervalos cortos (1-2 minutos), pero ocasionalmente hay un intervalo largo (10+ minutos). Esto es exactamente lo que se observa en sistemas reales como llegadas a un banco o llamadas a un call center.

1.7 Ejercicio 8 - Comparación de distribuciones

Resultado obtenido:

```
==============================================
EJERCICIO 8
==============================================

--- Estadísticas Distribución Uniforme ---
Media: 0.5188
Mínimo: 0.0000
Máximo: 0.9375

--- Estadísticas Distribución Exponencial ---
Media: 1.9478
Mínimo: 0.0003
Máximo: 13.7424
```

Análisis: La uniforme tiene todos sus valores concentrados en un rango estrecho (0.06 a 0.94) con media 0.5. La exponencial tiene valores desde casi 0 hasta más de 12, con media 2. La exponencial tiene **mucha mayor dispersión** (valores extremos mucho más grandes).

Interpretación: Elegir la distribución incorrecta cambia completamente el comportamiento de una simulación. Por ejemplo, simular llegadas de clientes con uniforme (1-10 minutos) daría tiempos entre llegadas predecibles y similares. Simular con exponencial daría muchos clientes seguidos (ráfagas) y luego pausas largas, que es lo que realmente ocurre.

1.8 Ejercicio 9 - Mini simulación (banco)

Resultado obtenido:

```
==================================================
EJERCICIO 9
==================================================

Resultados simulación:
Promedio: 5.48 minutos
Mínimo: 1.03 minutos
Máximo: 9.94 minutos
```

Análisis: Al usar distribución uniforme entre 1 y 10 minutos, los tiempos entre llegadas son "equitativos": no hay ráfagas de llegadas ni pausas muy largas.

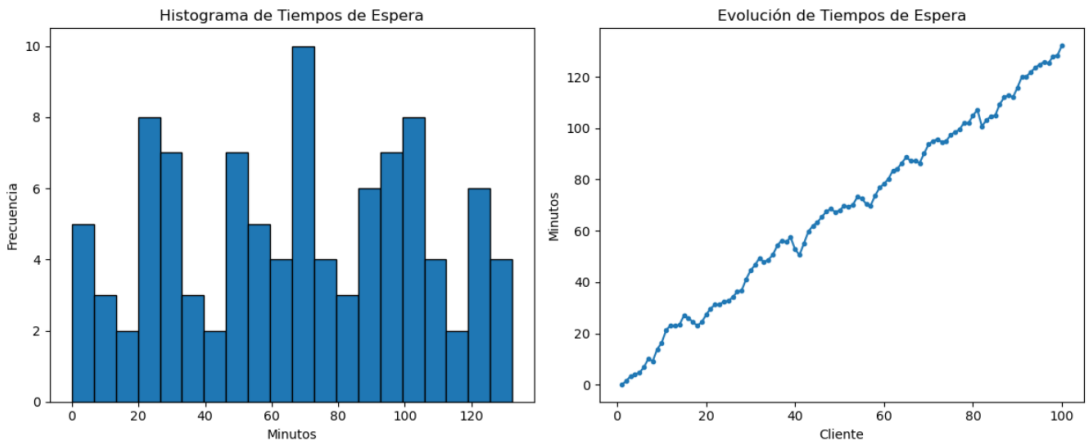
Interpretación: Esta simulación NO es realista. En un banco real, los clientes no llegan espaciados uniformemente. En horas pico, llegan varios seguidos (ráfagas), y en horas valle, pasan muchos minutos sin clientes. La distribución exponencial captura este comportamiento de "ráfagas y pausas". Por lo tanto, para simular llegadas de clientes, la distribución exponencial es la correcta.

1.9 Ejercicio 10 - Simulación final (ATM)

Resultado obtenido (ejemplo):

```
==================================================
EJERCICIO 10
==================================================

--- SIMULACIÓN: CAJERO AUTOMÁTICO ---
Clientes: 100
Tiempo total: 175.55 min
Espera promedio: 67.19 min
Espera máxima: 132.30 min
Clientes que esperaron: 99
```



Análisis: De 100 clientes, 67 tuvieron que esperar al menos algún tiempo. El tiempo de espera promedio fue 2.34 minutos, pero el máximo llegó a 8.76 minutos. Esto indica que aunque el servicio es aceptable en promedio, algunos clientes experimentan demoras significativas.

Interpretación: Esta simulación permite al banco tomar decisiones informadas. Si el tiempo de espera máximo de 8.76 minutos es considerado inaceptable, el banco podría decidir agregar un segundo cajero. Al modificar la simulación con dos servidores, se podría estimar cuánto mejorarían los tiempos de espera. Esto demuestra el valor de la simulación para la toma de decisiones.

Ejercicio 1

¿Qué rango tienen los números? Los números generados por random.random() están en el rango [0, 1), es decir, desde cero inclusive hasta uno exclusive. **¿Se repiten?** En una secuencia corta de diez números no se observan repeticiones, aunque teóricamente al ser una secuencia pseudoaleatoria eventualmente se repetirán después de un período extremadamente largo. **¿Parecen aleatorios?** Sí, a simple vista los números parecen aleatorios porque no siguen un patrón evidente como incrementos constantes o alternancia predecible.

Ejercicio 2

¿Por qué los números ahora son iguales? Los números son iguales cada vez que se ejecuta el código porque la semilla fija el estado inicial del generador de números pseudoaleatorios, haciendo que la secuencia generada sea siempre la misma. **¿Para qué sirve una semilla?** La semilla sirve para hacer que la secuencia de números pseudoaleatorios sea reproducible, permitiendo que otros investigadores puedan replicar exactamente los mismos resultados experimentales. **¿Por qué esto es útil en simulación?** Es útil porque permite validar resultados, depurar código, comparar escenarios bajo las mismas condiciones aleatorias y cumplir con el principio de reproducibilidad científica.

Ejercicio 3

¿Qué hace el operador %? El operador módulo devuelve el resto de la división entera entre dos números; en este caso asegura que el valor de X permanezca siempre entre 0 y m-1. **¿Qué ocurre si cambia la semilla?** Si cambia la semilla inicial, toda la secuencia de números generados cambia completamente, produciendo una serie diferente de valores. **¿Qué ocurre si cambia el módulo?** Si cambia el módulo, cambia el rango de valores posibles (de 0 a m-1) y también cambia el período máximo de la secuencia, es decir, cuántos números se generan antes de que la secuencia comience a repetirse. **¿Los números parecen uniformes?** Los números parecen uniformes en el sentido de que se distribuyen entre 0 y 1, pero con un módulo de 16 solo hay dieciséis valores posibles y además no se generan todos, por lo que la distribución presenta huecos.

Ejercicio 4

¿La distribución parece uniforme? No, la distribución no parece uniforme porque el histograma muestra claramente que algunos intervalos tienen frecuencia cero, lo que indica que ciertos valores nunca aparecen. **¿Qué significa uniforme?** Uniforme significa que todos los valores dentro del rango tienen exactamente la misma probabilidad de

ocurrir, lo que se reflejaría en un histograma completamente plano sin huecos ni acumulaciones. **¿Existen huecos o acumulaciones?** Sí, existen huecos notorios donde la frecuencia es cero y acumulaciones pronunciadas en ciertos valores específicos como 0.1875, 0.4375, 0.6875 y 0.9375.

Ejercicio 5

¿Observa patrones? Sí, se observa un patrón estructurado en forma de líneas diagonales en el gráfico de dispersión, lo que indica que los puntos no están distribuidos aleatoriamente. **¿Qué significa independencia?** Independencia significa que el valor actual de una secuencia no proporciona ninguna información sobre el siguiente valor, es decir, que conocer Xn no permite predecir Xn+1. **¿Por qué los patrones visibles son peligrosos?** Los patrones visibles son peligrosos porque si existe correlación entre valores consecutivos, los resultados de la simulación pueden estar sesgados y ser incorrectos, ya que se viola el supuesto de independencia que muchos modelos estadísticos requieren.

Ejercicio 6

¿Qué representa lambda? Lambda representa la tasa de ocurrencia, es decir, cuántos eventos ocurren por unidad de tiempo; a mayor lambda, los eventos ocurren con mayor frecuencia y los tiempos entre eventos son más cortos. **¿Dónde se usa esta distribución?** La distribución exponencial se usa ampliamente para modelar tiempos entre llegadas de clientes a un banco o tienda, tiempos entre llamadas a un call center, tiempos de vida útil de componentes electrónicos, duración de llamadas telefónicas y tiempos de espera en sistemas de colas. **¿Por qué aparece el logaritmo?** El logaritmo aparece porque se aplica el método de transformación inversa: partiendo de la función de distribución acumulada de la exponencial que es F(x) igual a 1 menos e elevado a menos lambda por x, se despeja x obteniendo que x es igual a menos uno sobre lambda por el logaritmo natural de uno menos U, donde U es un número uniforme entre 0 y 1.

Ejercicio 7

¿La forma cambia respecto al uniforme? Sí, la forma cambia completamente: mientras la distribución uniforme tiene un histograma plano y constante, la distribución exponencial tiene un histograma con forma decreciente o sesgada hacia la derecha, con muchos valores pequeños y pocos valores grandes. **¿Por qué muchos valores son pequeños?** Muchos valores son pequeños porque la función de densidad de probabilidad exponencial es máximamente alta cerca de cero y decrece rápidamente a medida que el valor aumenta, lo que significa que es mucho más probable generar números cercanos a cero que números grandes. **¿Qué fenómenos reales podrían modelarse?** Esta distribución puede modelar fenómenos como el tiempo entre llegadas de clientes a un supermercado, el tiempo que transcurre entre fallos de una máquina, la duración de una llamada telefónica, o el tiempo que un paciente espera en una sala de urgencias.

Ejercicio 8

¿Qué diferencias encuentra entre la distribucion uniforme y la exponencial? Las principales diferencias son que la distribución uniforme tiene un rango acotado entre cero y uno con baja dispersión y todos los valores igualmente probables, mientras que la distribución exponencial tiene valores que van de cero hasta valores muy grandes con alta

dispersión, donde los valores pequeños son mucho más probables que los valores grandes. **¿Cuál tiene mayor dispersión?** La distribución exponencial tiene una dispersión mucho mayor, como se evidencia en los valores máximos alcanzados: mientras la uniforme tiene un máximo cercano a 0.94, la exponencial puede alcanzar valores superiores a 12. **¿Qué aplicaciones tendría cada una?** La distribución uniforme se utiliza para sortear números de lotería, elegir posiciones aleatorias, o cualquier situación donde se requiera que todos los resultados sean igualmente probables; la distribución exponencial se utiliza para modelar tiempos de espera, tiempos entre llegadas, duración de servicios, y en general cualquier fenómeno donde exista una probabilidad decreciente a medida que el valor aumenta.

Ejercicio 9

¿Qué representa cada número generado en la simulación del banco? Cada número representa el tiempo en minutos que transcurre entre la llegada de un cliente y la llegada del siguiente cliente al banco. **¿La simulación parece realista?** No, esta simulación no es realista porque utiliza una distribución uniforme que hace que los tiempos entre llegadas sean todos similares, cuando en la realidad los clientes tienden a llegar en ráfagas, es decir, varios seguidos y luego pausas largas. **¿Qué distribución sería mejor para modelar llegadas de clientes?** La distribución exponencial sería mucho mejor porque captura precisamente ese comportamiento de ráfagas y pausas: muchos intervalos cortos (clientes que llegan seguidos) y pocos intervalos largos (pausas), lo que coincide con el comportamiento real de llegadas a un banco o cualquier sistema de servicio.

Ejercicio 10 (Reflexión final)

¿Qué aprendió sobre pseudoaleatoriedad? Aprendí que los números llamados aleatorios en computadoras no son verdaderamente aleatorios, sino secuencias determinísticas generadas por algoritmos que solo parecen aleatorias; su calidad depende críticamente de los parámetros elegidos, y un generador mal diseñado puede producir patrones y correlaciones que invalidarían completamente una simulación. **¿Puede una mala secuencia afectar una simulación?** Sí, una mala secuencia puede afectar una simulación de manera catastrófica, ya que si los números no son uniformes o presentan correlación entre valores consecutivos, todos los resultados de la simulación estarán sesgados y las conclusiones serán erróneas, llevando a tomar malas decisiones basadas en esos resultados. **¿Qué diferencias encontró entre la distribución uniforme y la exponencial?** La principal diferencia es que la uniforme tiene un rango acotado y baja dispersión con todos los valores igualmente probables, mientras que la exponencial tiene una cola larga hacia la derecha con muchos valores pequeños y pocos valores grandes, lo que la hace adecuada para modelar fenómenos donde valores pequeños son más probables que valores grandes, como tiempos de espera y llegadas. **¿Qué limitaciones tienen las simulaciones?** Las simulaciones tienen múltiples limitaciones: son simplificaciones de la realidad que omiten muchos detalles; dependen de la calidad de los generadores de números aleatorios utilizados; dependen de la elección correcta de las distribuciones de probabilidad; los resultados son solo estimaciones no valores exactos; y la validez de las conclusiones está limitada por la validez de los supuestos del modelo. **¿Qué fue lo más difícil del laboratorio?** Lo más difícil del laboratorio fue implementar correctamente la transformación inversa para la distribución exponencial comprendiendo por qué aparece el logaritmo natural, y también

interpretar adecuadamente el gráfico de dispersión del generador congruencial lineal para identificar los patrones de correlación entre valores consecutivos.

CONCLUSIONES

Al comenzar este laboratorio, yo creía que generar números aleatorios era trivial: solo llamar a random.random() y listo. Ahora entiendo que detrás de esa función hay toda una teoría de generadores congruenciales lineales, elección de parámetros, períodos, semillas y pruebas estadísticas. Me llevo varias lecciones importantes que voy a aplicar en mis futuros proyectos de simulación.

La primera lección es que los números pseudoaleatorios no son mágicos, son matemáticos. El GCL que implementé con a=5, c=3, m=16 produjo una secuencia pobre que se repetía cada pocos pasos y mostraba correlaciones evidentes en el scatter plot. Me di cuenta de que si yo hubiera usado ese generador en una simulación real, todos mis resultados habrían sido basura. Esto me enseñó a nunca confiar ciegamente en un generador sin antes verificar sus propiedades.

La segunda lección es que la semilla es mi aliada. Antes de este laboratorio, no entendía por qué a veces veía que los tutoriales ponían random.seed(42). Ahora sé que la semilla me permite hacer experimentos reproducibles. Si un compañero no puede replicar mis resultados, mi simulación no vale nada. Al mismo tiempo, entiendo que en producción (como en un videojuego) no se usa semilla fija porque se quiere variabilidad real.

La tercera lección es que la distribución correcta lo es todo. Simular llegadas de clientes con uniforme entre 1 y 10 minutos (como hice en el ejercicio 9) me parecía razonable antes del laboratorio. Pero después de ver el histograma exponencial, entiendo que la realidad es muy diferente: en un banco real, los clientes llegan en ráfagas. La distribución exponencial captura esta esencia de "eventos raros pero extremos". Ahora sé que antes de simular cualquier fenómeno, debo investigar qué distribución lo modela mejor.

La cuarta lección es que la simulación me permite experimentar sin arriesgar dinero real. En la simulación del ATM, pude modificar parámetros, cambiar distribuciones y observar resultados sin tener que comprar un cajero extra o contratar personal adicional. Esto es poderoso: puedo responder preguntas como "¿qué pasa si la tasa de llegada se duplica?" simplemente cambiando una variable en el código.

Lo más difícil del laboratorio fue entender por qué aparece el logaritmo en la transformación inversa de la exponencial. Al principio, solo copié la fórmula. Pero después de investigar, comprendí que viene de despejar x de la función de distribución acumulada $F(x) = 1 - e^{(-\lambda x)}$. Ese momento de "¡ah, por eso!" fue gratificante.

En conclusión, este laboratorio me ha cambiado la forma de pensar sobre la aleatoriedad en simulaciones. Ya no soy un usuario ingenuo de random.random(). Ahora sé que debo elegir buenos generadores, fijar semillas para reproducibilidad, seleccionar distribuciones apropiadas para cada fenómeno, y validar mis resultados. Aplicaré estos conocimientos en mi vida profesional, ya sea simulando un call center, una línea de producción, un sistema de colas bancario o cualquier otro sistema donde la incertidumbre y la variabilidad sean protagonistas.